DLDCA Lab Quiz 1

Week 5

September 8, 2026

Introduction

This is the first lab quiz. Read every instruction carefully, and note the marks for each task and subtask before starting so you can plan your time. Your code is autograded against the testcases described below, plus hidden testcases.

3 questions, 32 marks total (+ bonus marks).

If anything is unclear, trust sources in this order: **TA clarifications, then the testbench output and code comments, then this document.**

The following instructions are part of the problem statement and must be followed.

Documentation is in the lab quiz folder: slides for verilog and assembly and the same cheatsheets from the weekly labs. A reference table for assembly instructions, ABI and syscalls is at the end of this document.

Good luck!

Check your tools first

You need a working Verilog toolchain, a working assembly toolchain, and VS Code with its extensions. Check all three immediately and inform a TA if anything is missing or malfunctioning.

VS Code

- Go to **File** > **Auto Save** to turn on Auto Save.
- If the bottom-left corner shows **Restricted Mode**, click it and choose 'Trust' to enable extensions.
- Verify that there are **5 extensions** installed: `Vaporview`, `Verilog-HDL/SystemVerilog`, `x86 and x86_64 Assembly`, `vscode-pdf`, and `C/C++`.

Verilog

Check your Verilog toolchain: `iverilog` and `vvp`.

```
$ iverilog
iverilog: no source files.
...
$ vvp -V
Icarus Verilog runtime version [version]
```

Assembly

Check your assembly toolchain: `nasm` and `gcc`.

```
$ nasm -v
NASM version [version]
$ gcc --version
gcc ([linux-flavour] [version]) [version]
```

Submission

Check your submission toolchain: `check-cs231` and `submit-cs231`.

```
$ check-cs231
Enter you roll number to check your submission

Roll Number:
$ submit-cs231
Enter you roll number to submit

Roll Number:
```

Submission Procedure

Only the files marked `(TODO)` should be edited. Leave every other file (the testbenches, the expected-output files, etc.) exactly as given.

Your submission must contain exactly the following tree:

```
your-roll-no/
|- task1/
    |- task1.v (TODO)
    |- tb_task1.v
    |- expected_output.txt
|- task2/
    |- power_sum.s (TODO)
    |- digit_fn.o
    |- tests/          (provided testcases, do not edit)
|- task3/
    |- elevator.v (TODO)
    |- tb_task3.v
    |- expected_output_A_B.txt
    |- expected_output_C.txt
```

Build the submission with:

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Do not rename the resulting tarball!

`./build_archive.sh` only builds the tarball. It does **not** submit anything. **This is the actual submission procedure:**

1. **Create a folder `~/Desktop/submission_<rollNumInLowerCase>/` and move your tarball into it**
2. **Run `check-cs231` and type your roll number** when asked. This generates your final submission from that tarball. The only file getting added to your submission should be your tarball.
3. Call a TA and **Run `submit-cs231` to submit this zip.**

Make sure your code compiles when you submit.

Building and testing

Run everything from the `labquiz1` directory.

```
$ make task1               # Q1: run the testbench, check it, open the waveform
$ echo "123" | make task2  # Q2: build and run once on the piped-in string
$ make task2-check         # Q2: run every provided testcase
$ make task3               # Q3: run all tests
$ make check               # run all the autograders at once
$ make clean               # delete everything that was generated
```

Before submitting, run `make check` and make sure your code compiles. Pass as many tests as you can, however passing all of them doesn't guarantee full marks.

If a test keeps failing, try running `make clean` to delete stale files.

For Q1, `make` runs the testbench, saves the output to `task1/output.txt`, and diffs it against `task1/expected_output.txt`: an empty diff prints a green `PASS`, a non-empty one prints a red `FAIL` and `output.txt` shows which checks failed. The same idea applies to `task2-check` and `task3`.

Please read the testing and autograding sections in each task.

Q1. Verilog: Tere MUX MUX do nain [8 Marks]

`task1.v` has four empty modules that build on each other. Each one uses the previous, but you can stop after any part and still get credit for what works. Run `make task1` to build and test.

Note carefully what Verilog constructs are allowed and what are not. Note the hint in last part as well.

Part (a): `mux2` and `mux4` [1+2 Marks]

A 2:1 multiplexer picks one of two inputs. `mux2` has inputs `a`, `b`, a select bit `sel`, and output `y`: when `sel` is 0, `y` is `a`; when `sel` is 1, `y` is `b`.

Write `mux2` with a single `assign`. Do not use an `always` block or a register anywhere.

Now build a 4:1 multiplexer, `mux4`, which **must use your `mux2`**. It has a 4-bit input `d`, a 2-bit select `sel`, and output `y`, and should behave as `y = d[sel]`. Build it **structurally**: instantiate some number of `mux2` modules and wire them as a tree. You may have done this construction in CS230.

Do not use `assign`, an `always` block, or an indexed expression like `d[sel]` for `mux4`.

Part (b): `mux4_bus` [2 Marks]

`mux4` picks one of four bits. `mux4_bus` will pick one of four **bytes**. It has four 8-bit inputs `d0`, `d1`, `d2`, `d3`, a 2-bit select `sel`, and an 8-bit output `y`, with `y = d{sel}` (logically speaking).

Build it **structurally** using eight `mux4` instances, **one per bit**: bit `i` of `y` is chosen from bit `i` of the four inputs. You can either use a `generate` loop or write it out 8 times explicitly.

Do not use `assign` or `always` on `y` directly.

Part (c): `alu` [3 Marks]

Use `mux4_bus` to build an 8-bit ALU. It has inputs `a`, `b` (8-bit), `op` (2-bit), and output `y`:

op	operation	y
00	add	`a + b`
01	subtract	`a - b`
10	bitwise and	`a & b`
11	bitwise or	`a | b`

Arithmetic is done on 8-bit values. You can use wires to compute all four results, then use **one `mux4_bus`** instance to pick the answer.

Do not select the output with a `case`, an `always` block, or a conditional `assign` statement.

Hint. For 8-bit inputs (`wire [7:0] a, b`) the operators `+`, `-`, `&`, `|` are well-defined so writing `a + b`, `a - b`, `a & b`, `a | b` are allowed.

Testing

```
$ make task1
```

This compiles `task1.v` with `tb_task1.v`, runs the testbench, writes the transcript to `task1/output.txt`, and compares it against `task1/expected_output.txt`. You get a green `PASS` if they match exactly and a red `FAIL` otherwise; the summary at the end of `output.txt` tells you which parts passed. It also opens the waveform in VS Code. Full marks on the code needs every part passing and our hidden testcases.

Autograding

Marks are attached with subtask headings. The components will be tested on all possible input combinations and full marks awarded on correct behavior on all of them.

Please **follow the constraints** described in the problem statement when writing the code. Violating them for a module will result in **zero marks** for that module.

Q2. Assembly: Can I get your digits? [8 Marks]

Write the body of `power_sum` in `power_sum.s`. Everything outside the two `DO NOT EDIT` lines is provided and **must be untouched**: it reads a line from stdin, hands `power_sum` a pointer to that string (NULL-terminated, which means the last byte is a `NUL`, whose numeric value is `0`, unlike the digit '0' which has value `0x30`) in `rdi`, and prints whatever you return in `rax`.

Task: Write `power_sum` which will return sum of `digit_fn` applied to every valid digit of the number.

Explanation of `power_sum`: `power_sum` takes a string and reads every character as a byte. If it is a decimal digit `'0'-'9'`, convert it to its numeric value, call `digit_fn` with that value, and add the result to a running total. It stops at the first byte that is not a digit. If there are no valid digits, return 0. Return the total in `rax`.

Hint. You wrote `atoi` in inlab 6 and outlab 5 and did loops in inlab 5. The loop here has the same shape.

`digit_fn` is provided only as a compiled object, `digit_fn.o`:

```
digit_fn(uint64_t d{rdi}) -> uint64_t{rax}    ; 0 <= d <= 9
```

Note: You don't know what `digit_fn` is doing internally. Ensure that you save your data/registers accordingly. Caller-callee convention might be of some help :)

Testing

```
$ echo "123" | make task2
32
$ make task2-check
PASS -- task2/tests/01_input.txt
PASS -- task2/tests/02_input.txt
PASS -- task2/tests/03_input.txt
PASS -- task2/tests/04_input.txt
PASS -- task2/tests/05_input.txt
PASS -- task2/tests/06_input.txt
PASS -- task2/tests/07_input.txt
PASS -- task2/tests/08_input.txt
PASS -- task2/tests/09_input.txt
PASS -- task2/tests/10_input.txt
PASS -- task2/tests/11_input.txt
PASS -- task2/tests/12_input.txt
PASS -- task2/tests/13_input.txt
PASS -- task2/tests/14_input.txt
PASS -- task2/tests/15_input.txt
PASS -- task2/tests/16_input.txt
```

`make task2` will build `power_sum.s` against `digit_fn.o` and runs it once on whatever you pipe in.

`make task2-check` will build and run it on all tests. Tests are stored in `task2/tests/` as `NN_input.txt` with corresponding expected output `NN_output.txt` and reports `PASS` or `FAIL` for each.

Autograding

The following test case groups are *some of what* will be used for grading. Each group carries partial credit. **If your code passes all tests in a group, you get the marks shown below.** Final evaluation may consider additional test cases apart from the provided ones.

Group	Test cases	Marks
1	`""`, `" "`	1
2	`"0"`, `"1"`, `"5"`	1
3	`"12"`, `"121"`, `"999"`	1
4	`"0034"`, `"98070"`	1
5	`"512abc"`, `"95 15"`, `"9090..."` ("90" repeated 10 times)	2
6	Hidden test cases	2

Hardcoding any testcases will result in 0 marks.

Q3. Verilog: Bhavesh Returns In Doomsday Ft. Jean Grey [16 Marks + Bonus]

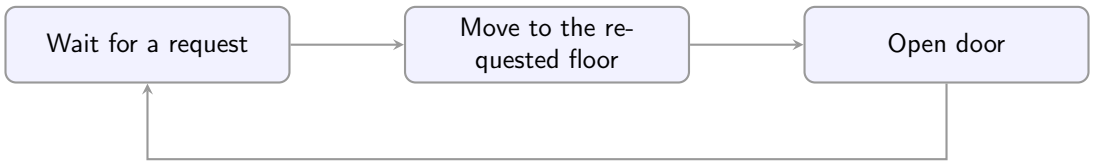
Bhavesh crossed the road. Now, he is stuck in a special building design by Dr. Doom to trap him.

The Doomstadt building has 7 floors, numbered `1` to `7`. He wants you to build the controller for its elevator as an FSM.

Floors are 3-bit numbers. The value `3'b000` is not a valid floor. We use it for representing "invalid" or "default" values.

Everything for this question goes in `elevator.v`, which contains two empty modules. The question has three parts. **If you do part A only you may score partial marks. Parts A and B together makes a working elevator, potential for full marks. Doing parts A, B and C, gives you potential for full + optional bonus marks.**

High-level idea. An elevator controller takes into account the current floor, a request from the passenger (internal request) and a request from the external user who wants to use the elevator (external request) and outputs a signal to the elevator motor to go up, down, or stop and also a signal to the door module to open or close the door. It repeats this logic in a loop:



The elevator also allows you to "store" one request while it is travelling. This will be clear in later subsections.

Part A: `floor_datapath` (combinational) [4 Marks]

The controller doesn't change `current_floor` itself. We need a separate module to represent the elevator motor: it takes the current floor and the two movement signals and produces the floor for the next cycle. The output floor should always be between `1` and `7` (inclusive).

```
module floor_datapath (
    input      [2:0] current_floor,
    input            move_up,
    input            move_down,
    output     [2:0] next_floor
);
```

- `move_up` high (and `move_down` low): `next_floor` is one above `current_floor`, but never past `7`.
- `move_down` high (and `move_up` low): `next_floor` is one below `current_floor`, but never below `1`.
- neither high, or both high: `next_floor` equals `current_floor`.
- `floor_datapath` should treat `current_floor` as a valid floor value; it does not need to handle the case of `current_floor = 3'b000` specially at all.

Use only `assign` or a combinational `always @(*)` block. Clocks or registers are not allowed..

Part B: Bhavesh's elevator controller [12 Marks]

```
module elevator (
    input            clk,
    input            rst,           // active-high, asynchronous
    input      [2:0] internal_req,
    input      [2:0] external_req,
    input      [2:0] start_floor,   // current_floor starts here after reset
    input      [1:0] jean_grey,     // used only in Part C
    output reg [2:0] current_floor,
    output           move_up,
    output           move_down,
    output           door_open
);
```

Instantiate your `floor_datapath` inside `elevator`: store your `current_floor` in a register fed by its `next_floor` output (loaded with `start_floor` on reset), and wire your `move_up` / `move_down` into it.

Two request sources

The elevator takes floor requests from two places: `internal_req` from a passenger inside, and `external_req` from someone waiting on a floor. Internal request has higher priority.

Request either carries the floor to visit, or `3'b000` for no request. Each request lasts a single clock cycle.

States

The `state` register is 2-bit wide. **it is given to you in the template**:

```
    localparam IDLE        = 2'd0;
    localparam MOVING_UP   = 2'd1;
    localparam MOVING_DOWN = 2'd2;
    localparam DOOR_OPEN   = 2'd3;

    reg [1:0] state;
    reg [2:0] target_floor;   // floor currently being travelled to (0 = none)
    reg [1:0] door_timer;
```

Holding a request

If the elevator is busy and someone makes a request, it needs to be latched in. This register is already declared for you in the template:

```
    reg [2:0] pending_request;   // 0 = nothing waiting
```

Rules:

1. If `internal_req` and `external_req` arrive in the same cycle, `internal_req` takes priority.

2. If the elevator is ***busy*** (`MOVING_UP`, `MOVING_DOWN`, or `DOOR_OPEN`), save the first request in `pending_request`. Ignore later requests until the pending request is loaded into `target_floor`.
3. **(Optimization)** If the elevator is `IDLE` and `pending_request` is empty, put a new request **directly into** `target_floor`. This lets the elevator start moving on the *next* clock cycle, instead of waiting an extra cycle to latch the request first.

Movement

- In `IDLE`, if `pending_request` is not empty, load it into `target_floor` and clear `pending_request`. Do this only in `IDLE`. Otherwise, if a fresh request arrives, use that request directly as the new `target_floor` (as discussed in 3). Compare this new target with `current_floor`:
  - if the target is above `current_floor`, go to `MOVING_UP`;
  - if the target is below `current_floor`, go to `MOVING_DOWN`;
  - if they are equal, go to `DOOR_OPEN`.
- In `MOVING_UP`, set `move_up = 1` while `current_floor < target_floor`. When they become equal, set `move_up = 0` and go to `DOOR_OPEN`. `MOVING_DOWN` works the same way with `move_down`.
- On the cycle where `current_floor == target_floor`, the state is still `MOVING_UP` / `MOVING_DOWN`, but the corresponding movement output is already 0.
- `move_up` and `move_down` are 0 in all other states, and they are never both 1.

Doors

- `DOOR_OPEN` lasts exactly 4 clock cycles (`door_timer` counts `0,1,2,3`), then the state returns to `IDLE`.
- `door_open` is 1 only in `DOOR_OPEN` state.
- `target_floor` **is always reset to** `3'b000` when `DOOR_OPEN` is entered.

Allowed transitions

A summary of what is allowed:

```
inputs                                    | curr_state  -> next_state
------------------------------------------------------------------------
nothing waiting, no fresh request         | IDLE        -> IDLE
target loaded, target above current floor | IDLE        -> MOVING_UP
target loaded, target below current floor | IDLE        -> MOVING_DOWN
target loaded, target is the current floor| IDLE        -> DOOR_OPEN
------------------------------------------------------------------------
current_floor < target_floor              | MOVING_UP   -> MOVING_UP
current_floor == target_floor             | MOVING_UP   -> DOOR_OPEN
current_floor > target_floor              | MOVING_DOWN -> MOVING_DOWN
current_floor == target_floor             | MOVING_DOWN -> DOOR_OPEN
------------------------------------------------------------------------
door_timer < 3                            | DOOR_OPEN   -> DOOR_OPEN
door_timer == 3                           | DOOR_OPEN   -> IDLE
```

Reset

`rst` is active-high and asynchronous (`always @(posedge clk or posedge rst)`). Upon resetting:

- state is `IDLE`
- `current_floor` is `start_floor`
- `target_floor` and `pending_request` are `3'b000`
- the door timer is 0
- all three outputs are 0

Part C (Optional, Bonus): Help Jean Grey [Bonus]

Jean Grey is still sending signals. This time, Bhavesh needs your help figuring them out...

This part is optional and does not affect the requirements for Parts A and B. **Make sure your code passes part A and B tests when you submit!**

`jean_grey` is a 2-bit signal, sending one symbol every clock cycle.

- If the last 3 consecutive cycles' symbols together have an equal number of 0s and 1s (i.e. exactly 3 of each, among the 6 bits), that is a **trigger**.
- **Trigger-Based Reset:** When the trigger condition is met, the elevator controller must reset on that exact clock edge. Specifically, `state`, `target_floor`, `pending_request`, and `door_timer` must reset exactly as they would under a normal `rst` signal. This must occur in the exact same cycle that the triggering `jean_grey` bits are received. Consequently, the output signals (`move_up`, `move_down`, and `door_open`) must immediately become 0 so the elevator stops instantly and does not move during this cycle, while `current_floor` remains unchanged. (Please refer to the expected output for clarification.)

You cannot add a new value to `state`. And, you should use only **one extra register** (no constraint on additional wiring/logic).

Initialization. Assume that, for the first 3 cycles after reset, `jean_grey` sends `2'b00` so you don't have to worry about that edge case.

Running the tests

```
$ make task3
```

The testbench runs Part A's tests, then Part B, then Part C. Each Part B test item starts by placing the elevator in the exact configuration needed for that test, so the test items are independent of one another. A bug in one behaviour should therefore not affect the results of unrelated test items.

Each test item prints its test number (matching the grading table) and a `PASS` / `FAIL` verdict, for every time it is tested. For example:

```
TIME (1ns after posedge)       | STATE/OUTPUTS (after that posedge)  | INPUTS (at that posedge)
-------------------------------------------------------------------------------------------------
t=  56                         | MOVING_DN  floor=5 up=0 dn=1 door=0 | int=0 ext=0 jg=00
t=  66                         | IDLE       floor=3 up=0 dn=0 door=0 | int=0 ext=0 jg=00
t=  76                         | IDLE       floor=3 up=0 dn=0 door=0 | int=0 ext=0 jg=00
  [Test Item 1] reset #1       | PASS
t=  86                         | IDLE       floor=3 up=0 dn=0 door=0 | int=0 ext=0 jg=00
t=  96                         | MOVING_UP  floor=4 up=1 dn=0 door=0 | int=0 ext=0 jg=00
  [Test Item 1] reset #2       | PASS
```

The output is split into two halves at a long `###...###` delimiter line: everything above is Parts A and B, everything below is Part C. Each half is graded against its own *expected-output file*, `task3/expected_output_A_B.txt` / `expected_output_C.txt`, and reported as its own large colored `PASS` / `FAIL` banner.

You can read the line-by-line output or open in VaporView for clarity and debugging.

Minimal worked example (IDLE → MOVING_UP). Elevator is `IDLE` at floor 3, nothing pending, `external_req = 5` appears:

- Cycle 1: `target_floor` becomes 5 (acc to 3, since nothing was pending); since $5 > 3$, `state` becomes `MOVING_UP` and `move_up = 1`.
- Cycle 2: `current_floor` becomes 4.
- Cycle 3: `current_floor` becomes 5, equal to `target_floor` — `move_up` drops to 0 this same cycle, and the next state is `DOOR_OPEN`.
- Cycles 4–7: state is `DOOR_OPEN` (`door_timer` counts 0,1,2,3); the `DOOR_OPEN` → `IDLE` transition is decided at cycle 7's edge, so `state` reads `IDLE` starting cycle 8.
- Cycle 8: back in `IDLE`. `target_floor` is already 0 (it was cleared the moment `DOOR_OPEN` was entered, at cycle 4) and nothing is pending, so the elevator just waits for the next request.

Autograding

Part A

Partial marks for correct behaviour on each combination of `move_up` and `move_down`, including boundary cases.

Part B

The following behaviours will be used for grading. Each carries partial credit. The last item carries more credit because it tests the complete behaviour over a longer sequence.

Test Item Number	Behaviour	Marks
1	Reset	1
2	IDLE → UP/DOWN and movement signals	1
3	Reaching target and entering DOOR_OPEN immediately	1
4	Door opening timing	1
5	Target reset on door open	1
6	Internal request priority	1
7	Latching pending request while busy	2
8	Direct request optimization in IDLE	1
9	Long sequence with multiple requests	3

Part C (Bonus)

The following behaviour will be tested using various sequences of `jean_grey` inputs and different configurations of the elevator. These configurations should be achievable if Parts A and B are implemented correctly. **If code for part A or B is faulty then zero marks will be awarded for bonus.**

Table 1: Some basic x86-64 Assembly Instructions

Instruction	Operands	Effect
`mov`	`reg, reg/*mem/imm`	`reg` ← `reg/*mem/imm`
`mov`	`*mem, reg`	`*mem` ← `reg`
`movzx / movsx`	`reg, reg/*mem`	`reg` ← `reg/*mem`, zero-/sign-extended
`lea`	`reg, *mem`	`reg` ← address of `*mem`
`add`	`reg, reg/*mem/imm`	`reg` ← `reg` + `reg/*mem/imm`
`add`	`*mem, reg/imm`	`*mem` ← `*mem` + `reg/imm`
`sub`	`reg, reg/*mem/imm`	`reg` ← `reg` − `reg/*mem/imm`
`sub`	`*mem, reg/imm`	`*mem` ← `*mem` − `reg/imm`
`inc`	`reg/*mem`	`reg/*mem` ← `reg/*mem` + 1
`dec`	`reg/*mem`	`reg/*mem` ← `reg/*mem` − 1
`neg`	`reg/*mem`	`reg/*mem` ← −`reg/*mem` (Two's complement)
`not`	`reg/*mem`	`reg/*mem` ← ¬`reg/*mem` (One's complement/bitwise)
`shl / shr`	`reg/*mem, imm/cl`	Logical shift left / right by `imm` or `cl` bits
`sar`	`reg/*mem, imm/cl`	Arithmetic shift right (preserves sign bit)
`imul`	`reg, reg/*mem/imm`	`reg` ← `reg` × `reg/*mem/imm`
`imul`	`*mem, reg/imm`	`*mem` ← `*mem` × `reg/imm`
`div`	`reg`	`rax` ← `(rdx:rax)`/`reg` `rdx` ← `(rdx:rax)`%`reg`
`and`	`reg, reg/*mem/imm`	`reg` ← `reg` ∧ `reg/*mem/imm`
`and`	`*mem, reg/imm`	`*mem` ← `*mem` ∧ `reg/imm`
`or`	`reg, reg/*mem/imm`	`reg` ← `reg` ∨ `reg/*mem/imm`
`or`	`*mem, reg/imm`	`*mem` ← `*mem` ∨ `reg/imm`
`xor`	`reg, reg/*mem/imm`	`reg` ← `reg` ⊕ `reg/*mem/imm`
`xor`	`*mem, reg/imm`	`*mem` ← `*mem` ⊕ `reg/imm`
`cmp`	`reg/*mem, reg/imm`	Sets flags based on subtraction: `reg/*mem` - `reg/imm`. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
`test`	`reg/*mem, reg/imm`	Sets flags based on bitwise AND: `reg/*mem` ∧ `reg/imm`. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
`jmp`	`label`	Unconditional jump to label
`je / jne`	`label`	Jump if (not) equal / zero
`ja / jae`	`label`	Jump if above / above-or-equal (unsigned)
`jb / jbe`	`label`	Jump if below / below-or-equal (unsigned)
`jg / jge`	`label`	Jump if greater / greater-or-equal (signed)
`jl / jle`	`label`	Jump if less / less-or-equal (signed)
`cmov`*cc*	`reg, reg/*mem`	`reg` ← `reg/*mem` if *cc* holds, else unchanged.
`set`*cc*	`reg/*mem`	`reg/*mem` ← 1 if *cc* holds, else 0.
`syscall`		Performs a syscall with syscall number stored in `rax` Arguments passed in other registers starting from `rdi`
`push`	`reg`	Pushes the value in `reg` onto the stack
`pop`	`reg`	Pops the value on top of the stack onto `reg`
`call`	`label`	Calls the function called `label`
`ret`		Pops the return address from stack and jumps to it

Table 2: System V AMD64 ABI — Calling Convention Quick Reference

Rule	**Detail**
Argument order	`rdi, rsi, rdx, rcx, r8, r9` (1st through 6th argument)
Return value	`rax` (for a 64-bit return value)
Caller-saved (volatile)	`rax, rcx, rdx, rsi, rdi, r8, r9, r10, r11`
Callee-saved (preserved)	`rbx, rbp, r12, r13, r14, r15, rsp`
Raw `syscall` argument order	`rax` = syscall number, then `rdi, rsi, rdx, r10, r8, r9`
More rules	Before a `call` to a libc function, ensure the stack is 16-byte aligned. One way: `and rsp, -16` at the start of your function, then maintain alignment through matched `push`/`pop` pairs.

Table 3: Limited Syscall Table

Syscall	`rax`	Arguments (`rdi, rsi, rdx`)	Returns (`rax`)
`read`	0	`fd, buf, count`	bytes read (0 = EOF)
`write`	1	`fd, buf, count`	bytes written
`exit`	60	`status`	does not return